

Qizeng xu – Refactored the core codebase, applying key techniques such as function extraction, variable renaming, and conditional simplification.

Wenshan yan – Developed and executed unit tests, and used the coverage tool to ensure consistent functionality post-refactoring.

Jingran zhang – Conducted code reviews, suggested improvements for readability and clarity, and ensured adherence to clean code principles.

Zixin wang – Composed this report, summarised the refactoring process, and prepared the final deliverables.

Summary of Changes

The original main.py file had several issues, including ambiguous variable names, large and complex methods, deeply nested conditionals, and repetitive logic. Our goal was to restructure the code to improve clarity, modularity, and maintainability.

We reorganized the logic into smaller, more focused functions and replaced vague identifiers with descriptive names. Repetitive operations were centralised, and conditionals were simplified to better express intent.

We began by using Extract Function to break down large methods—such as generate_report()—into smaller, reusable functions. This made the code more modular and easier to test. Next, we applied Rename Variable to replace vague or unclear variable names with more descriptive ones, significantly improving code readability.

To simplify complex conditional logic, we adopted the Decompose Conditional technique, which helped replace nested if-else structures with clearer, more structured logic blocks. We also applied Inline Variable to eliminate unnecessary temporary variables that added no value to the code's logic or clarity.

Finally, we used Remove Duplication to centralize repeated logic for tax calculation and shipping fee determination. This reduced redundancy and made the codebase easier to maintain and extend in the future.

Behavioural Consistency Verification

To confirm that functionality remained consistent after refactoring:

We executed all unit tests using coverage run -m unittest tests/test_main.py

An HTML report was generated via coverage html to inspect coverage

Coverage report: 91%

Files

Functions

Classes

coverage.py v7.8.0, created at 2025-05-02 12:20 +0100

File ▲	statements	missing	excluded	coverage
main.py	49	8	0	84%
tests__init__.py	0	0	0	100%
tests\test_main.py	51	1	0	98%
Total	100	9	0	91%

coverage.py v7.8.0, created at 2025-05-02 12:20 +0100

Outputs from generate_report() were manually compared before and after refactoring tests passed successfully, and the observable behaviour remained unchanged

Benefits and Drawbacks

Benefits:

Increased code readability and maintainability

Improved modularity, making testing and debugging easier

Reduced complexity through smaller, more focused functions

Better separation of business logic

Drawbacks:

A higher number of smaller methods may feel excessive for small-scale scripts

Further enhancements may be needed for edge case coverage and cohesion

Reflection

The refactoring process proved to be highly valuable. It not only improved the technical structure of the code but also made the logic easier to follow and maintain. Despite the additional time required to refactor and validate changes, the long-term benefits justify the effort.

This exercise reinforced the value of clean code practices and highlighted how thoughtful design can significantly enhance software quality.